LIVE CLASS NOTES

Planning & reflection

5 Nov 2026 · 19:49–22:03 · 134 min · Rahul Tiwari

Overview

This session (Session 3) focused on building a PR Review Agent using LangGraph, covering hands-on implementation of GitHub API integration, tool creation, and agentic design patterns. The instructor, Rahul, guided participants through connecting a LangGraph-based agent to GitHub, creating Python tools for reading and posting PR data, and introduced multiple agentic architectural patterns applicable in real-world AI engineering scenarios.

Key Concepts or Theories

- **Tool Calling Loop**: LLMs iteratively call tools, collect outputs, and re-evaluate until the task is complete
- **ReAct Pattern**: Agents reason (think) and then act (call tools or APIs) in a dynamic, non-sequential loop
- **Plan and Execute Pattern**: A planner agent generates step-by-step plans; a separate executor agent carries them out; a replanner verifies alignment
- **Prompt Chaining**: A deterministic, sequential flow where each LLM output feeds into the next stage with no dynamic routing
- **Routing Pattern**: A manager agent analyzes input and delegates tasks to specialized sub-agents
- **Evaluate and Optimize (Reflection)**: A generator agent produces output; a critic/evaluator agent provides feedback in a loop until quality constraints are met
- **Multi-Agent Design Patterns**: Centralized, decentralized, and hybrid architectures for agent-to-agent communication

Important Questions Raised

- How does an agent decide when to stop calling tools?
- Why does an agent perform read operations freely but hesitate on write operations (e.g., posting GitHub comments)?
- How do multiple agents communicate with each other in a multi-agent system?
- How do we manage token costs when passing large codebases to agents?
- When should you use ReAct vs. Plan-and-Execute vs. Prompt Chaining patterns?
- How can free or open-source models (e.g., Hugging Face) be substituted for paid models like GPT-4o?

Key Takeaways and Summary of Learning Objectives

- A PR Review Agent was built using LangGraph, connecting to GitHub via API tokens and reading PR details, file lists, and file contents through custom Python tools
- Python functions are converted into LangGraph-compatible tools using the `@tool` decorator from LangChain, which requires proper docstrings
- Agents operate in a loop: they receive input, select tools, collect outputs, re-evaluate, and terminate when the goal is achieved
- Agents freely perform read operations but require explicit prompt instructions or human intervention to perform write operations (e.g., posting comments)
- Agentic patterns (ReAct, Plan-Execute, Prompt Chaining, Routing, Reflection) are not mutually exclusive and can be combined based on the problem statement
- Token management is critical: avoid passing entire codebases; use techniques like AST parsing or summarization to reduce context size
- Multi-agent systems should be designed with clear agent roles, limited tool sets per agent, and appropriate inter-agent communication edges

---

Topic 1: Building the PR Review Agent with GitHub Integration

The session began with a hands-on demonstration of building a PR Review Agent. Rahul walked participants through setting up GitHub API credentials, creating a personal access token with repository access, and storing credentials in a `.env` file. The agent was designed to accept a PR number as input and autonomously read PR details, list changed files, retrieve file contents, and post review comments back to GitHub.

Four Python tool functions were created:

- **getPRDetails**: Reads the PR title, description, author, state, and branch
- **getPRFiles**: Lists all files changed in a given PR
- **getFileContent**: Retrieves the full content of a specific file (limited to ~4,000 characters to manage token costs)
- **postPRComment**: Posts a review comment back to the GitHub PR

Each function was decorated with `@tool` from LangChain, converting them into agent-accessible tools. A LangGraph state was defined containing a list of messages and the PR number. The agent node was built with a system prompt instructing it to act as a code reviewer, produce structured feedback (what looks good, what was found, suggestions, and approval/rejection verdict), and use the provided tools.

A conditional edge (`should_continue`) was implemented to determine whether the agent should call more tools or terminate. The graph was compiled and invoked with an initial message referencing a live PR (PR #14) created during the session.

Key observation: The agent successfully read files and produced a review verdict but did not autonomously post comments. This was identified as expected agent behavior — agents are cautious about write operations and require explicit prompt instructions or deterministic graph edges to enforce such actions.

Relevant Q&A

Question (dishant varshney): In the file content tool, you limited content to 4,000 characters. In a real project with many files, how do we handle this?

Answer (Rahul): Use an AST (Abstract Syntax Tree) parser to identify only the changed functions or code snippets within a file, rather than passing the entire file. This is a common industry approach. There are also 3–4 other strategies to manage token costs in code review scenarios.

Question (sneha sharma): What is the difference between binding tools to the LLM directly and adding a tool node in the graph?

Answer (Rahul): Binding tools to the LLM gives the agent access to call those tools. Adding a tool node in the graph is for visualization and orchestration purposes — it makes the tool execution step explicit in the graph diagram. Both are needed: the LLM binding enables tool calling; the graph node represents it structurally.

Question (S. Valli): Where is the LLM model defined and how is the API key provided?

Answer (Rahul): The LLM is defined inside the agent node function. The API key is loaded from the `.env` file at the start of the notebook, as covered in Session 1. The model (e.g., GPT-4o Mini) is initialized and then bound to the list of tools using `bind_tools`.

Question (Shiva Shankar PP): On what basis does the agent decide it has enough information to stop calling tools?

Answer (Rahul): The agent compares the accumulated list of messages (all tool outputs refed back into the agent) against the goal stated in the system prompt. When the agent determines the goal is fulfilled, it generates a final response instead of a tool call, which triggers the termination condition.

Topic 2: Agentic Design Patterns

Rahul introduced and explained multiple agentic patterns that practitioners use in industry, emphasizing that no single pattern is universally correct — the choice depends on the problem statement.

Tool Calling Loop: The foundational pattern. An LLM receives input, decides whether tools are needed, calls them, receives outputs, and loops until the task is complete. This is the basis for all more complex patterns.

ReAct (Reason + Act): Extends the tool calling loop by explicitly including a reasoning step in the prompt. The agent is instructed to "think, then act." Structurally similar to the tool calling loop but with prompt-level guidance to reason before selecting an action. Most commonly cited in interviews.

Plan and Execute: Three agents collaborate — a Planner generates a full task list, an Executor carries out tasks one by one, and a Replanner verifies whether the execution aligns with the original plan. Suitable for complex, multi-step tasks such as code generation platforms (e.g., similar to Vercel v0.dev or Claude's artifact generation). The executor internally may use a ReAct loop.

Prompt Chaining: A fully deterministic, sequential pipeline where each LLM output feeds into the next stage. No dynamic routing. Suitable for well-defined workflows such as data validation pipelines, document classification, or report generation where all steps are known in advance.

Routing Pattern: A manager agent analyzes incoming input at a high level and routes it to one or more specialized sub-agents. Sub-agents can communicate with each other via bidirectional edges or through a "talk-to-agent" tool that re-activates the manager. Suitable for PR review systems with specialized security, coding standards, and performance agents.

Evaluate and Optimize (Reflection): A generator agent produces output; an evaluator/critic agent assesses it against defined constraints and provides feedback. The loop continues until quality thresholds are met. Distinct from Plan-and-Execute: the replanner checks plan alignment, while the evaluator critiques output quality.

Multi-Agent Design Architectures:

- **Centralized**: One manager agent routes to multiple sub-agents; sub-agents report back to the manager
- **Decentralized**: Agents communicate freely with each other without a central manager, using shared state or tool-based handoffs
- **Hybrid**: Combines centralized routing with decentralized agent-to-agent communication within teams

Rahul shared a handbook containing case studies (customer support, algorithmic trading, digital platforms) with architecture diagrams, agent rosters, and step-by-step flow definitions implemented in LangGraph.

Relevant Q&A

Question (sneha sharma): Is the ReAct pattern the same as the tool calling loop?

Answer (Rahul): They are ~90% similar structurally. The key difference is that ReAct adds an explicit "think then act" instruction at the prompt level. Code and graph structure remain the same; only the prompt changes.

Question (Sharad Keshari): How do agents communicate back and forth — for example, if a sub-agent needs to return control to the manager?

Answer (Rahul): Sub-agents can be given a "talk-to-agent" tool (a Python function) that, when called, updates the shared state and re-activates the manager node. This enables bidirectional communication without hardcoding the flow in the graph edges.

Question (Shiva Shankar PP): Is there a constraint on how many agents to use, similar to hardware load limits?

Answer (Rahul): The primary constraint is token context length. As more messages are fed back into an agent, performance degrades. Best practices: use 3–4 agents grouped by specific responsibility, limit tools per agent, and use summarization to compress historical context rather than passing the full message history.

Question (Sharad Keshari): Can free models like Hugging Face be used instead of paid OpenAI models?

Answer (Rahul): Yes. Rahul committed to adding a notebook to the Session 3 folder demonstrating how to call Hugging Face models as a drop-in replacement with a single line change.

Topic 3: Industry Practices and Career Guidance

Rahul shared broader guidance on how to approach AI engineering professionally, emphasizing that the goal of the bootcamp is not just assignment completion but building skills applicable to real jobs and interviews.

Key points:

- Focus on understanding how to apply agentic patterns to real business problems, not just completing exercises
- In production, deterministic flows are preferred over fully autonomous agents because autonomous systems become harder to debug and maintain at scale
- When selecting an AI model, match the model to the task complexity: use free/small models or traditional ML for simple classification; use larger models only when reasoning or generation is genuinely required
- Agents built in Python can be exposed as APIs and consumed by any technology stack (Java via Spring AI, or any other language)
- The HireFlow project in the Coding Ninjas repository demonstrates a job-matching agent that compares resumes against job postings
- Ollama (local models) can be connected to LangGraph agents as an alternative to cloud-based models

Relevant Q&A

Question (Arnab Halder): Can an agent be used to match a resume against LinkedIn or Naukri job postings?

Answer (Rahul): Yes. The HireFlow project in the Coding Ninjas repository does exactly this. Clone it, add your resume, and run it against job descriptions copied from LinkedIn or Naukri.

Question (S. Valli): Can LangGraph agents be used in Java-based projects?

Answer (Rahul): LangGraph is Python-native. For Java, Spring AI provides similar capabilities. Alternatively, build the agent in Python and expose it as a REST API, which any language can consume.

Actionable Next Steps / Assignments

- Review and run the updated Session 3 code in the GitHub repository (previously missing code has been committed)
- Read through the Multi-Agent Systems handbook shared in the Session 3 Notes folder, focusing on case studies and architecture diagrams
- Fix the PR Review Agent so that the `postPRComment` tool is reliably called by the agent (hint: update the system prompt to explicitly instruct the agent to use the tool, or add a deterministic graph edge)
- Explore the AST (Abstract Syntax Tree) parser library for handling large codebases in code review agents
- Follow AI engineering practitioners on LinkedIn to supplement learning with real-world perspectives
- Complete any pending course practicals using the session materials and AI assistance as needed
- No class on Saturday due to Diwali — use the time to go through the handbook and set up any pending installations

---

Supplemental Resources

- **Multi-Agent Systems Handbook**: Shared by Rahul in the Session 3 Notes folder of the course GitHub repository. Contains case studies (customer support, algorithmic trading, digital platforms), architecture diagrams, agent roster templates, and design pattern comparisons
- **HireFlow Project**: Available in the Coding Ninjas GitHub repository. A resume-to-job-posting matching agent. Clone locally, add resume, and run against job descriptions from LinkedIn or Naukri
- **Hugging Face Models Notebook**: Rahul committed to adding a notebook to the Session 3 folder demonstrating how to replace OpenAI models with free Hugging Face models in LangGraph agents
- **Rahul Tiwari on LinkedIn**: Search "Rahul Tiwari EY" on LinkedIn to follow the instructor's posts on AI engineering practices and industry insights
- **Spring AI Documentation**: Recommended for participants working in Java who want to build agent-based systems in a Java stack

- **Vercel v0.dev**: Referenced as an example of a Plan-and-Execute agentic system for web application generation from prompts

---